

Scripts explanation

- **Training** adjusts a model's weights (internal numbers) using examples. We use already-trained models for predictions, so no training is needed here.
- **Fine-tuning** is extra training to adapt a trained model to a specific task. We use the models as provided, so no fine-tuning is needed here.

Two examples, two places to run a model

	Sentiment example	OpenAI example
Task	Classify a short English sentence.	Generate an answer to a question.
Model runs	On the CPU in your Colab runtime.	On OpenAI's servers.
Result	POSITIVE or NEGATIVE	Generated text.
Access needed	Internet for setup and model download. No API key for this model.	Internet, an authorised API key, model access and available API quota.

What is a runtime? It is the computer session executing your code. In standard hosted Colab, that computer is a Google-hosted virtual machine, not your laptop. Here, a model running "locally" means inside that runtime. [1]

Run the notebook successfully

1. Open the notebook in Colab and save your own copy. The default CPU runtime is enough.
2. Run cells from top to bottom using the play button or **Shift+Enter**. Wait for each installation to finish before running its example.
3. When a prompt appears, type your input and press Enter. If you start a new runtime, run the installation and setup cells again.

Execution order matters: Python runs the cells you execute, in that order. Moving a cell or reading it does not run it. Within these code cells, statements execute from top to bottom.

Read the Python

You see	It means
<code>%pip -q install ...</code>	Install packages in the notebook runtime. -q reduces installation messages.
<code>from ... import ...</code>	Make a specific library feature available in your code.
<code>name = value</code>	Store a value under a variable name. <code>input()</code> reads text. <code>print()</code> displays it.

1. Classify sentiment

The notebook uses a DistilBERT model fine-tuned on English movie-review sentiment. It chooses **POSITIVE** or **NEGATIVE**. It has no neutral label. [2]

First run `%pip -q install transformers torch`. Then run the example below.

NOTEBOOK CODE | Comments omitted for readability

```
from transformers import pipeline
model = pipeline(
    "sentiment-analysis",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
    device="cpu",
)
sentence = input("Write a short sentence in English: ")
result = model(sentence, truncation=True)[0]
print("Prediction:", result["label"])
```

Follow the code

Expression	What happens
<code>pipeline(...)</code>	Builds a pipeline containing the tokenizer and trained model. <code>model</code> is the variable holding that pipeline.
<code>device="cpu"</code>	Runs the model on the runtime's CPU. No GPU is required.
<code>input(...)</code>	Pauses for a sentence and stores the typed text in <code>sentence</code> .
<code>model(sentence, truncation=True)</code>	Makes a prediction. Text longer than the model's token limit is shortened. Some content can be lost.
<code>[0]</code> and <code>["label"]</code>	The pipeline returns a list. <code>[0]</code> selects its first result. <code>["label"]</code> reads the label from that result. [3]

A numerical example: from text to a label

Input: **I loved this movie!** The token IDs and scores below are invented for teaching. This is a simplified illustration of inference: using a trained model.

Token	i	loved	this	movie	!
ID	12	45	8	93	6

How the percentages are created: the trained model gives each label a raw score, for example `NEGATIVE = 1.2` and `POSITIVE = 4.7`. A calculation called softmax changes those scores into probabilities that add up to 1.00.

In this toy example, softmax gives `NEGATIVE = 0.03` and `POSITIVE = 0.97`, or about 3% negative and 97% positive. The pipeline chooses **POSITIVE** because `0.97 > 0.03`. The percentage is model confidence, not a guarantee.

Pipeline result: `[{"label": "POSITIVE", "score": 0.97}]` `[0]` selects the result. `["label"]` gives `POSITIVE`.

2. Ask a model through an API

An **API** lets programs request a service. Your code sends a question and instructions to GPT-4.1 mini, which uses learned patterns to generate an answer one token at a time. [4, 5]

First run `%pip -q install openai`. Use an authorised [API key](#) with available quota. API usage is billed to that account. Ask your teacher if you do not have access.

NOTEBOOK CODE | Comments omitted for readability

```
from getpass import getpass
from openai import OpenAI
client = OpenAI(api_key=getpass("Enter your OpenAI API key: "))
question = input("Ask the AI a question: ")
response = client.responses.create(
    model="gpt-4.1-mini",
    instructions="You are a friendly teacher. Answer in three short sentences.",
    input=question,
    max_output_tokens=200,
)
print(response.output_text)
```

Follow the request

Expression	What happens
<code>getpass(...)</code>	Hides typing when supported. The key remains in memory. If warned that typing is visible, stop and ask for help. [6]
<code>OpenAI(...)</code>	Creates an API client using your key. Creating the client does not generate an answer.
<code>responses.create(...)</code>	Sends an HTTPS request to OpenAI. <code>model=</code> selects the model. <code>input=question</code> supplies your question.
<code>instructions=...</code>	Requests a friendly teaching style and three short sentences. It guides the answer. Exact wording and format can vary.
<code>max_output_tokens=200</code>	Caps generated output tokens, not words. A token is a piece of text. The response may end early if it reaches this limit.
<code>response.output_text</code>	Reads the generated text from the response object. <code>print(...)</code> displays that text.

One question per run: The code sends only the current question and instructions. Earlier questions and answers are not included. [7] No web-search tool is used. Check facts against reliable sources.

Keep the key private. Enter it only in the hidden prompt. Do not put it in code, print it, or share it in screenshots. Each rerun of this example asks for the key again and can incur API usage.

Official references

[1] Colab FAQ | [2] DistilBERT model card | [3] Transformers pipelines | [4] OpenAI text generation
[5] GPT-4.1 mini | [6] Python getpass | [7] Conversation state